the request body:

```
provider
    .uponReceiving('a request to add a new movie')
 .withRequest({
    method: 'POST',
    body: YOUR_BODY_HERE
    path: '/movies',
  })
```

When you've written the additional tests, don't forget to update the `--consumer-app-version` as part of the `publish:contract` script, and run the following command in your terminal to run the test:

```
npm run test:consumer
npm run publish:contract
```

If you encounter any problems with your tests and want to generate the contract found in WebConsumer-MoviesAPI.json, you can always delete the json file. The contract will be generated again after running the `npm run test:consumer` command.

Once the contract has been published to PactFlow, run the command to run the provider verification test. If you get stuck at any point, check out the main branch of the original GitHub project for assistance at https://github.com/mdcruz/pact-js-example/tree/main, or check out appendix C for the answers.

Summary

- The focus of provider contract tests is to verify the contract generated by their consumers.
- Contract testing shouldn't cover all the provider's business rules because contract testing isn't the same as functional testing and should be handled by a separate set of tests on the provider codebase.
- Provider states allow data providers to define the state they need to be in to verify the interaction from the consumer contract successfully.
- Pact provides a set of functionalities for the provider verification through the `Verifier` interface.